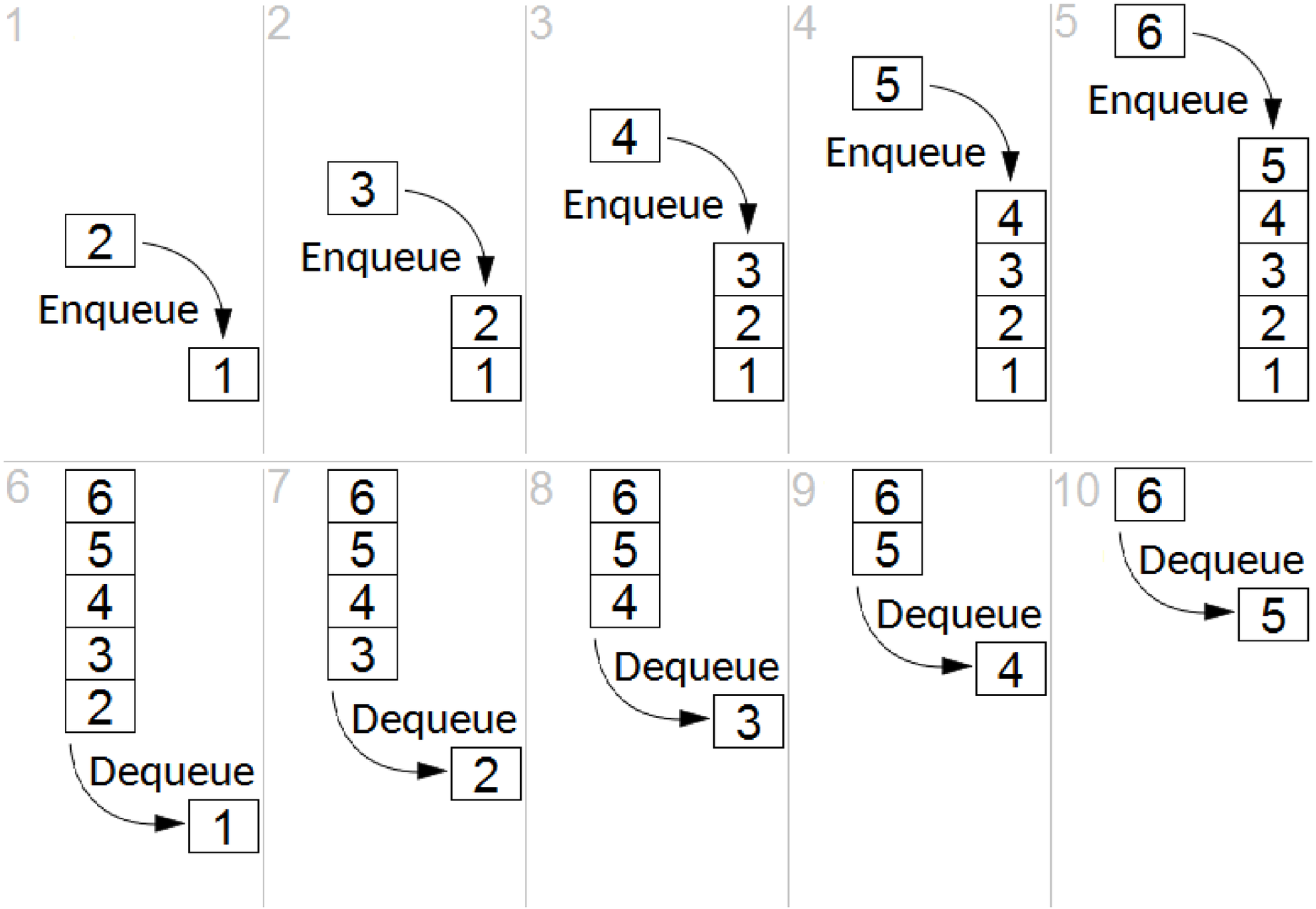
Python

FILA

O que é FILA ?

- Pense em uma fila de supermercado ou de um banco.
- A primeira pessoa a chegar é a primeira a ser atendida.
- Em ciência da computação, uma Fila (ou Queue, em inglês) é exatamente isso: uma estrutura de dados linear que segue o princípio FIFO (First-In, First-Out).


1
2
Enqueue
1
2
3
Enqueue
2
1
3
4
Enqueue
3
2
1
4
5
Enqueue
4
3
2
1
5
6
Enqueue
5
4
3
2
1
6
6
5
4
3
2
Dequeue
1
7
6
5
4
3
Dequeue
2
8
6
5
4
Dequeue
3
9
6
5
Dequeue
4
10
6
Dequeue
5

Principais Operações de uma Fila:

- **Enqueue (Enfileirar):** Adiciona um novo elemento ao final da fila. (append)
- **Dequeue (Desenfileirar):** Remove e retorna o elemento que está no início da fila. (popleft)
- **Peek / Front (Espiar)**: Retorna o elemento do início da fila sem removê-lo. (fila[0])
- **isEmpty (Está Vazia?):** Verifica se a fila não contém elementos. (if not fila)
- **isFull (Está Cheia?):** Verifica se a fila atingiu sua capacidade máxima (relevante para filas com tamanho fixo).

Como Implementar Filas em Python?

- Embora seja possível usar uma lista (list) do Python para simular uma fila, essa não é a abordagem mais eficiente.
    - Remover um elemento do início de uma lista (com **list.pop(0))** é uma operação lenta, pois todos os outros elementos precisam ser deslocados.
- A forma recomendada e mais eficiente é usar a classe deque (pronuncia-se "deck", de double-ended queue) do módulo collections.
    - “deck” foi otimizada para adições e remoções rápidas em ambas as extremidades.

Exemplo 1

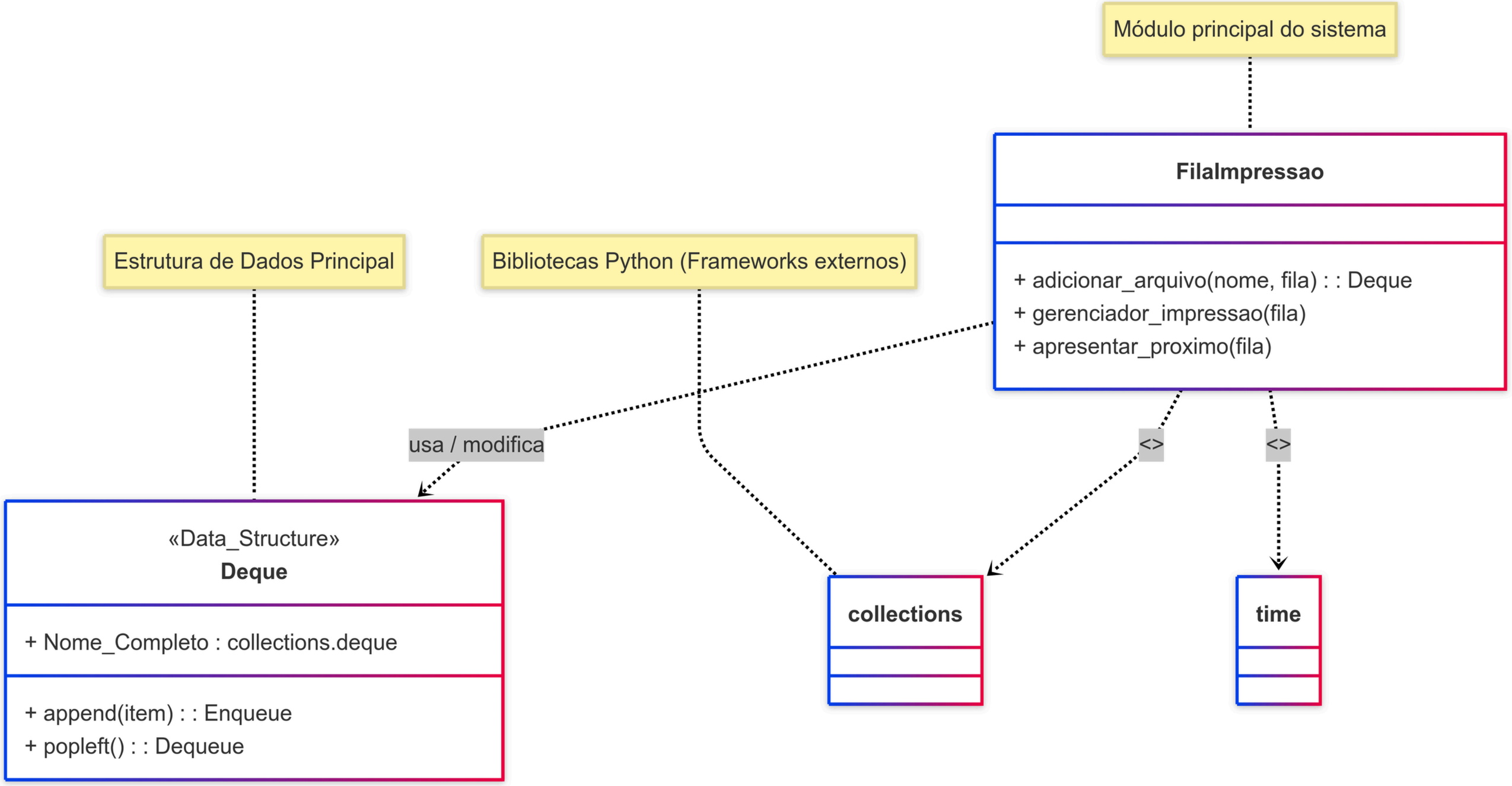
Gerenciador de Fila de Impressão

Problema

- Vamos simular um sistema simples que gerencia documentos em uma fila de impressão.
- Documentos são adicionados à fila e impressos na ordem em que foram recebidos.

Conceito Aplicado:

- Este é o caso de uso mais clássico de uma fila. Usaremos **collections.deque** para gerenciar os documentos.
  - **append()** será nosso "**enqueue**" e
  - **popleft()** será nosso "**dequeue**".


Módulo principal do sistema
FilaImpressao
+ adicionar_arquivo(nome, fila) : : Deque
+ gerenciador_impressao(fila)
+ apresentar_proximo(fila)
Estrutura de Dados Principal
Bibliotecas Python (Frameworks externos)
usa / modifica
<>
<>
«Data_Structure»
Deque
+ Nome_Completo : collections.deque
+ append(item) : : Enqueue
+ popleft() : : Dequeue
collections
time

Arquivo fila.py

```
import collections
import time


def adicionar_arquivo(nome_arquivo, fila_de_impressao=None):
    """
    Simula uma fila de impressão onde documentos (arquivos) são adicionados
    e processados em ordem. Retorna a fila atualizada ao final.
    """
    # Cria uma fila vazia usando deque, se não existir
    if not fila_de_impressao:
        fila_de_impressao = collections.deque()

    print("Iniciando o gerenciador de impressão.")
    # Adicionando documentos à fila em intervalos de tempo
    print(f"Adicionando à fila: {nome_arquivo}")
    fila_de_impressao.append(nome_arquivo) # Enqueue
    print("\nFila de impressão atual:", list(fila_de_impressao))
    print("-" * 30)
    return fila_de_impressao
```

Arquivo
fila.py

```
def gerenciador_impressao(fila_de_impressao=None):
    # Processando (imprimindo) os documentos da fila
    while fila_de_impressao:
        documento_atual = fila_de_impressao.popleft() # Dequeue
        print(f"Imprimindo: {documento_atual}...")
        time.sleep(2) # Simula o tempo de impressão
        print(f"'{documento_atual}' impresso com sucesso!")


    print("\nTodos os documentos foram impressos. Fila vazia.")


def apresentar_proximo(fila_de_impressao):
    if fila_de_impressao:
        print(f"Próximo documento a ser impresso: {fila_de_impressao[0]}")
    else:
        print("A fila de impressão está vazia.")
```

Arquivo app.py

```
from fila import adicionar_arquivo, gerenciador_impressao, apresentar_proximo


def main():
    print("Bem-vindo ao Gerenciador de Impressão!")
    fila_de_impressao = []


    while True:
        print("\nEscolha uma opção:")
        print("1. Adicionar arquivo à fila de impressão")
        print("2. Executar impressao")
        print("3. Apresentar próximo documento")
        print("4. Sair")


        opcao = input("Digite o número da opção desejada: ")
```

Arquivo
app.py

```
        if opcao == "1":
            nome_arquivo = input("Digite o nome do arquivo a ser adicionado: ")
            fila_de_impressao = adicionar_arquivo(nome_arquivo, fila_de_impressao)
        elif opcao == "2":
            gerenciador_impressao(fila_de_impressao)
        elif opcao == "3":
            apresentar_proximo(fila_de_impressao)
        elif opcao == "4":
            print("Saindo do programa. Até logo!")
            break
        else:
            print("Opção inválida. Tente novamente.")



if __name__ == "__main__":
    main()
```

Explicando

- Este exemplo é a demonstração do princípio **FIFO (o primeiro a entrar é o primeiro a sair).**
- O primeiro documento enviado para a fila é, obrigatoriamente, o primeiro a ser impresso.
- A fila garante a justiça e a ordem no processo. A estrutura de dados ideal para isso é a Fila (Queue).

Exemplo 2

Atendimento ao Cliente em uma Central de Suporte

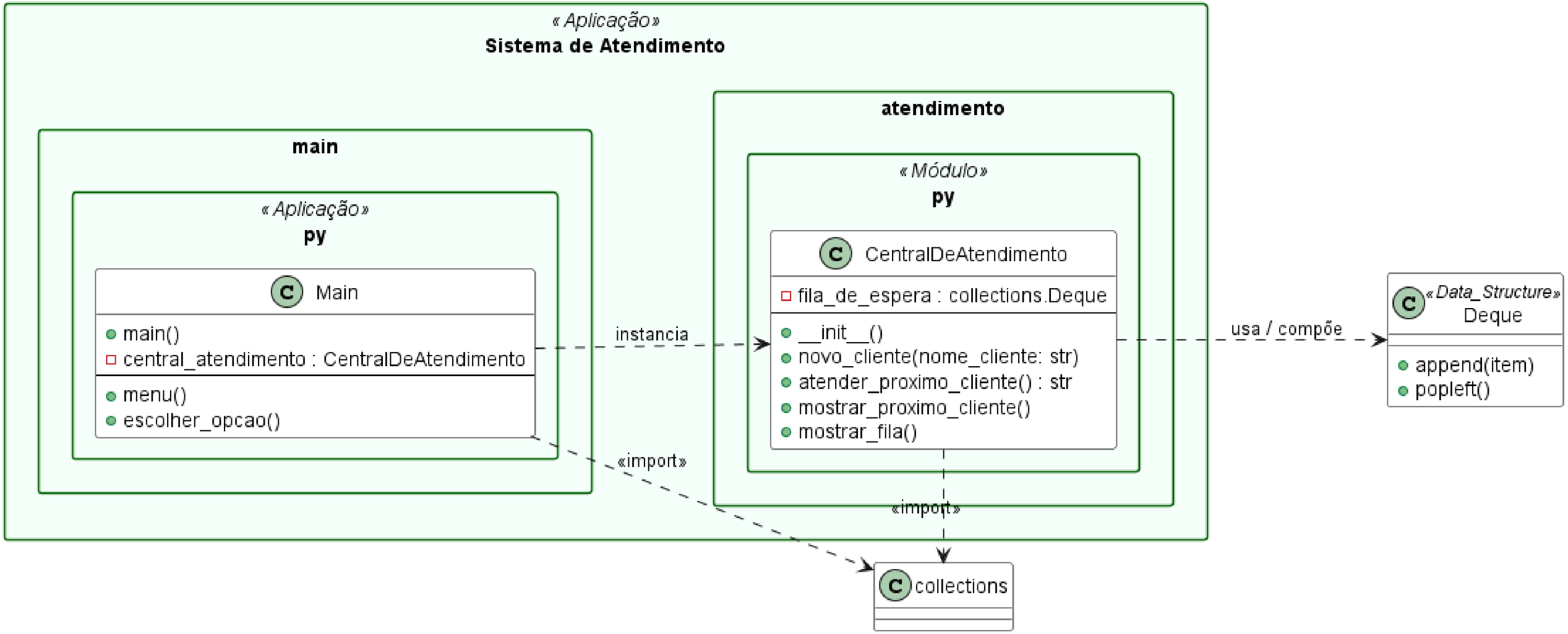
Problema

- Simular uma fila de atendimento onde clientes entram em uma fila para serem atendidos por um agente. O sistema deve atender os clientes por ordem de chegada.

Conceito Aplicado

- Similar ao primeiro exemplo, mas focando em um cenário de serviços. A estrutura FIFO garante que o atendimento seja justo, seguindo a ordem de chegada.

Comparativo de Sistemas com Fila - Diagrama de Classes



«Aplicação»: Módulo principal do sistema
«Módulo»: Componente lógico
«Framework»: Biblioteca padrão Python
«Data_Structure»: Estrutura de dados utilizada

Arquivo atendimento.py

```python
import collections


class CentralDeAtendimento:
    def __init__(self):
        self.fila_de_espera = collections.deque()
        print("Central de Atendimento iniciada. Nenhum cliente na fila.")

    def novo_cliente(self, nome_cliente):
        """Cliente entra na fila de espera."""
        print(f"Cliente '{nome_cliente}' entrou na fila.")
        self.fila_de_espera.append(nome_cliente) # Enqueue
        self.mostrar_fila()

    def atender_proximo_cliente(self):
        """Agente atende o primeiro cliente da fila."""
        if not self.fila_de_espera:
            print("Não há clientes na fila para atender.")
            return None

        cliente_atendido = self.fila_de_espera.popleft() # Dequeue
        print(f"--> Agente está atendendo '{cliente_atendido}'.")
        self.mostrar_fila()
        return cliente_atendido
```

Arquivo atendimento.py

```
    def mostrar_proximo_cliente(self):
        """Mostra o próximo cliente na fila."""
        if not self.fila_de_espera:
            print("Não há clientes na fila para atender.")
        else:
            print(f"Próximo cliente na fila: {self.fila_de_espera[0]}\n")



    def mostrar_fila(self):
        """Mostra o estado atual da fila de espera."""
        if self.fila_de_espera:
            print(f"Fila atual: {list(self.fila_de_espera)}\n")
        else:
            print("Fila está vazia.\n")
```

Arquivo
app.py

```python
from atendimento import CentralDeAtendimento
def main():
    central_atendimento = CentralDeAtendimento()

    while True:
        print("\nEscolha uma opção:")
        print("1. Novo cliente")
        print("2. Atender próximo cliente")
        print("3. Apresentar o próximo a ser atendido")
        print("4. Sair")
        opcao = input("Digite o número da opção desejada: ")
        if opcao == "1":
            nome_cliente = input("Digite o nome do cliente: ")
            central_atendimento.novo_cliente(nome_cliente)
        elif opcao == "2":
            central_atendimento.atender_proximo_cliente()
        elif opcao == "3":
            central_atendimento.mostrar_proximo_cliente()
        elif opcao == "4":
            print("Saindo do programa. Até logo!")
            break
        else:
            print("Opção inválida. Tente novamente.")

if __name__ == "__main__":
    main()
```

Resumo e Próximos Passos

- **Entenda o Conceito:** O mais importante é solidificar o princípio FIFO.
- **Use collections.deque:** Para implementações de filas em Python, deque é a ferramenta certa. Use append() para enfileirar e popleft() para desenfileirar.
- **Pratique:** Tente modificar os exemplos acima ou crie seus próprios cenários. Que tal um sistema de buffer para streaming de vídeo ou um simulador de "round-robin" para alocação de tarefas?
- **Pratique:** A fila é fundamental em algoritmos como a Busca em Largura (BFS), que explora um grafo camada por camada. Além disso, é essencial em sistemas que gerenciam recursos na ordem de chegada (princípio FIFO), como filas de impressão, agendadores de processos em sistemas operacionais e no gerenciamento de requisições em servidores.

Fim

Maromo